

SPEC — Sistema Interno de Requisição de Materiais (MVP)

Documento de especificação para desenvolvimento por agente de IA. Stack obrigatória: **HTML + CSS + JS puro + Supabase**, hospedado no **GitHub Pages**. Não usar frameworks (React, Vue, Bootstrap, Tailwind), nem backend próprio.

1. Contexto e Objetivo

A empresa perde dinheiro por ausência de controle sobre insumos internos: resmas de papel somem, toners de impressora secam no estoque e mouses/teclados são solicitados em duplicidade.

Objetivo do MVP: garantir que toda saída de material do estoque passe por uma requisição digital aprovada, com saldo atualizado e histórico rastreável.

Regra de ouro: nenhum material sai do estoque sem requisição aprovada e baixa registrada.

Critérios de sucesso (após 1 mês de piloto)

- 100% das saídas de material possuem registro (quem, quando, quanto).
- O Admin consegue consultar o histórico completo de qualquer material.
- Estoque nunca fica negativo.

2. Stack (NÃO DESVIAR)

Camada	Tecnologia
Frontend	HTML5, CSS3, JavaScript (ES Modules, sem build step)
Backend/DB/Auth	Supabase (Postgres + Auth + RLS)
Hospedagem	GitHub Pages
Bibliotecas externas	Apenas CDN do @supabase/supabase-js v2

Conta Supabase

- Projeto já criado pelo usuário (o agente deve pedir SUPABASE_URL e SUPABASE_ANON_KEY se não fornecidas).
- Auth por e-mail + senha. Confirmação de e-mail pode ficar desabilitada em dev.

3. Modelo de Dados

3.1 Tabela materiais

Coluna	Tipo	Regras
id	uuid PK	gen_random_uuid()
nome	text NOT NULL	único (case-insensitive)
unidade	text NOT NULL	ex.: “un”, “resma”, “cx”

Table 2 – continued

Coluna	Tipo	Regras
saldo	integer NOT NULL DEFAULT 0	nunca negativo
estoque_min	integer NOT NULL DEFAULT 0	usado para alerta visual
ativo	boolean NOT NULL DEFAULT true	desativação lógica (não apagar)
criado_em	timestampz DEFAULT now()	

3.2 Tabela requisicoes

Coluna	Tipo	Regras
id	uuid PK	
material_id	uuid FK → materiais	
solicitante_id	uuid FK → auth.users	preenchido automaticamente
quantidade	integer NOT NULL	> 0
status	text NOT NULL DEFAULT 'pendente'	pendente aprovada recusada entregue
justificativa	text	opcional no MVP
criado_em	timestampz DEFAULT now()	
resolvido_em	timestampz NULL	preenchido na aprovação/recusa
resolvido_por	uuid FK → auth.users NULL	quem aprovou/recusou/entregou

3.3 SQL de criação

```
create table materiais (
  id uuid primary key default gen_random_uuid(),
  nome text not null unique,
  unidade text not null,
  saldo integer not null default 0 check (saldo >= 0),
  estoque_min integer not null default 0 check (estoque_min >= 0),
  ativo boolean not null default true,
  criado_em timestampz not null default now()
);

create table requisicoes (
  id uuid primary key default gen_random_uuid(),
  material_id uuid not null references materiais(id),
  solicitante_id uuid not null references auth.users(id),
  quantidade integer not null check (quantidade > 0),
  status text not null default 'pendente'
  check (status in ('pendente', 'aprovada', 'recusada', 'entregue')),
  justificativa text,
  criado_em timestampz not null default now(),
  resolvido_em timestampz,
  resolvido_por uuid references auth.users(id)
);
```

```
create index idx_requisicoes_status on requisicoes(status);
create index idx_requisicoes_solicitante on requisicoes(solicitante_id);
```

3.4 Função RPC entregar_requisicao (baixa atômica)

A baixa no estoque **deve** ser uma RPC no Postgres (transação atômica), não duas chamadas no client:

```
create or replace function entregar_requisicao(p_requisicao_id uuid)
returns void language plpgsql security definer as $$
declare
    req record;
begin
    select * into req from requisicoes where id = p_requisicao_id for update;
    if not found then raise exception 'Requisição não encontrada'; end if;
    if req.status <> 'aprovada' then
        raise exception 'Só é possível entregar requisições aprovadas';
    end if;
    update materiais set saldo = saldo - req.quantidade
        where id = req.material_id and saldo >= req.quantidade;
    if not found then
        raise exception 'Saldo insuficiente';
    end if;
    update requisicoes set status = 'entregue',
        resolvido_em = now(), resolvido_por = auth.uid()
        where id = p_requisicao_id;
end $$;
```

3.5 Row Level Security (OBRIGATÓRIO)

```
alter table materiais enable row level security;
alter table requisicoes enable row level security;

-- helper: perfil admin (definir após criar o primeiro usuário admin)
create or replace function is_admin()
returns boolean language sql stable as $$
    select coalesce((auth.jwt() -> 'user_metadata' ->> 'perfil') = 'admin', false)
$$;

-- materiais: todos autenticados leem; só admin escreve
create policy "leitura materiais" on materiais
    for select to authenticated using (true);
create policy "escrita admin materiais" on materiais
    for all to authenticated using (is_admin()) with check (is_admin());

-- requisições: usuário lê as próprias; admin lê todas
create policy "leitura proprias reqs" on requisicoes
    for select to authenticated
    using (solicitante_id = auth.uid() or is_admin());

-- criação: qualquer autenticado, material ativo, saldo >= quantidade
create policy "criar requisicao" on requisicoes
```

```

for insert to authenticated
with check (
  solicitante_id = auth.uid()
  and status = 'pendente'
  and exists (
    select 1 from materiais m
    where m.id = material_id and m.ativo and m.saldo >= quantidade
  )
);

-- update: apenas admin (aprovar/recusar/entregar via RPC)
create policy "admin gerencia reqs" on requisicoes
  for update to authenticated using (is_admin()) with check (is_admin());

```

Nota para o agente: após criar o primeiro usuário no Supabase Auth, defina `user_metadata.perfil = "admin"` manualmente no dashboard do Supabase. Todos os demais usuários são solicitantes (padrão).

4. Estrutura de Arquivos

```

/
├─ index.html          # tela de login
├─ painel.html         # lista de requisições (minhas / todas para admin)
├─ nova-requisicao.html # formulário de nova requisição
├─ materiais.html      # CRUD de materiais + entrada de estoque (só admin)
├─ js/
│   ├─ supabase.js     # cria e exporta o client
│   ├─ auth.js         # guard de sessão + função isAdmin()
│   └─ ui.js           # helpers: toast/alert, formatação de data, status badge
├─ css/
│   └─ style.css        # CSS puro, mobile-first

```

4.1 js/supabase.js

```

import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2/+esm';

export const supabase = createClient(
  'SUPABASE_URL',      // TODO: substituir
  'SUPABASE_ANON_KEY'  // TODO: substituir
);

```

4.2 js/auth.js

- `requireAuth()`: redireciona para `index.html` se não houver sessão.
- `requireAdmin()`: redireciona para `painel.html` se o usuário não for admin.
- `getPerfil()`: lê `user.user_metadata.perfil` (default `'solicitante'`).
- `Expore logout()` que chama `supabase.auth.signOut()` e redireciona ao login.

5. Telas e Comportamentos

5.1 index.html — Login

- Formulário: e-mail + senha.
- `supabase.auth.signInWithPassword()`; em sucesso, redireciona para `painel.html`.
- Se já houver sessão válida ao abrir a página, redireciona direto ao painel.
- Mostrar mensagem de erro amigável em caso de credenciais inválidas.

5.2 painel.html — Lista de Requisições

- **Solicitante:** vê apenas suas requisições, ordenadas por `criado_em` desc.
- **Admin:** vê todas, com filtro por status (`pendente`, `aprovada`, `recusada`, `entregue`, `todas`).
- Cada linha exibe: material (nome), quantidade + unidade, solicitante (para admin), status (badge colorido), data de criação.
- Botão “Nova requisição” → `nova-requisicao.html`.
- **Admin, requisições pendente:** botões “Aprovar” e “Recusar” (com `confirm()`).
 - Aprovar → `update status='aprovada', resolvido_em=now(), resolvido_por=auth.uid()`.
 - Recusar → idem com `'recusada'`.
- **Admin, requisições aprovada:** botão “Registrar entrega” → chama RPC `entregar_requisicao(id)`. Em erro (ex.: saldo insuficiente), exibir mensagem retornada pelo Postgres.

5.3 nova-requisicao.html

- Select de materiais: apenas `ativo = true`, ordenados por nome.
- Exibir saldo atual de cada material (no select ou ao selecionar).
- Campo quantidade (integer, min 1, máx = saldo atual — validar no JS e confiar no RLS como barreira final).
- Campo justificativa (textarea, opcional).
- Ao enviar: insert em `requisicoes` com `solicitante_id = auth.uid()`, status `pendente`.
- Feedback de sucesso e redirecionamento ao painel.

5.4 materiais.html (só admin — `requireAdmin()`)

- Tabela com: nome, unidade, saldo, estoque mínimo, badge de alerta se `saldo <= estoque_min`, `ativo`.
- **Cadastrar/Editar:** modal ou seção com formulário (nome, unidade, saldo inicial, estoque mínimo).
- **Entrada de estoque:** botão “Entrada” por material → prompt/modal com quantidade → `update saldo = saldo + qtd` (RPC simples ou update direto com `is_admin()`).
- **Desativar/Reativar:** toggle de `ativo` (nunca apagar registros).
- Saldo **nunca** editável manualmente para baixo — saída só via requisição.

6. Regras de Negócio (resumo para o agente)

1. Saldo nunca negativo — garantido por check (`saldo >= 0`), RLS e RPC.
2. Só materiais ativos e com saldo suficiente podem ser requisitados.
3. Só admin aprova, recusa, entrega e gerencia materiais.
4. Não há exclusão física de dados (soft delete via `ativo`, e requisições são imutáveis após `entregue`).
5. Auditoria mínima: `criado_em`, `resolvido_em`, `resolvido_por` em toda requisição.

7. UX / UI

- **Mobile-first** (almoxarife pode usar celular/tablet no estoque).
- CSS puro, variáveis para cores; sem framework.
- Badges de status: **pendente** (amarelo), **aprovada** (azul), **entregue** (verde), **recusada** (vermelho).
- Mensagens de erro amigáveis (traduzir erros do Postgres para PT-BR quando possível).
- Feedback visual em toda ação (botão desabilitado durante submit, mensagem de sucesso/erro).

8. Deploy (GitHub Pages)

1. Criar repositório público (ou privado com GitHub Pro) e subir os arquivos.
2. Em **Settings** → **Pages**, publicar a branch `main` (root).
3. Configurar no Supabase: **Authentication** → **URL Configuration**:
 - Site URL: `https://<usuario>.github.io/<repo>/`
 - Redirect URLs: `idem`.
4. Substituir `SUPABASE_URL` e `SUPABASE_ANON_KEY` em `js/supabase.js` (chave anon é pública por design — a segurança está no RLS).

9. Fora de Escopo (não implementar no MVP)

- Dashboards, gráficos e relatórios exportáveis
- Notificações por e-mail
- Múltiplos itens por requisição (carrinho)
- Controle de validade/lotes de toner
- Integração com AD/LDAP
- Qualquer framework JS ou build step

10. Ordem de Implementação Sugerida

1. SQL: tabelas, RPC, RLS (validar no SQL Editor do Supabase)
2. `supabase.js` + `auth.js` + tela de login
3. `materiais.html` (CRUD + entrada) — para ter dados de teste
4. `nova-requisicao.html`
5. `painel.html` com ações de admin
6. Polimento de UX e testes com 3–5 usuários piloto